PROJECT REPORT

Automated Network Request Management

A ServiceNow scoped low-code application built with App Engine Studio

Platform	ServiceNow (Personal Developer Instance)
Team ID	
Team Members	
Institution	
Date	2 November 2026

Team ID, member names and institution were not provided and are left blank.

Contents

1. INTRODUCTION **3**

1.1 Project Overview . . . 3

1.2 Purpose . . . 3

2. IDEATION PHASE **3**

2.1 Problem Statement . . . 3

2.2 Empathy Map Canvas . . . 3

2.3 Brainstorming . . . 4

3. REQUIREMENT ANALYSIS **4**

3.1 Customer Journey Map . . . 4

3.2 Solution Requirement . . . 5

3.3 Data Flow Diagram . . . 5

3.4 Technology Stack . . . 6

4. PROJECT DESIGN **7**

4.1 Problem Solution Fit . . . 7

4.2 Proposed Solution . . . 8

4.3 Solution Architecture . . . 8

5. PROJECT PLANNING & SCHEDULING **8**

5.1 Project Planning . . . 8

6. FUNCTIONAL AND PERFORMANCE TESTING **9**

Functional testing results . . . 9

6.1 Performance Testing . . . 9

7. RESULTS **10**

7.1 Output Screenshots . . . 10

8. ADVANTAGES & DISADVANTAGES **14**

Advantages . . . 14

Disadvantages and limitations . . . 14

9. CONCLUSION **14**

10. FUTURE SCOPE **14**

11. APPENDIX **15**

Source Code (if any) . . . 15

Dataset Link . . . 15

GitHub & Project Demo Link . . . 15

1. INTRODUCTION

1.1 Project Overview

Automated Network Request Management is a ServiceNow scoped low-code application built in ServiceNow App Engine Studio on a hosted Personal Developer Instance (PDI). It manages network service requests (new connections, existing connections and related services) through a Service Catalog item in the Service Portal and Flow Designer flows.

Requesters submit a Network Request form. A flow stores the request in a custom table, sends an acknowledgement email, asks for approval and updates the record with the approved or rejected outcome. A separate custom table tracks task work, and a second flow sends a resolution email when a task is completed. The application demonstrates a structured request lifecycle; it is a prototype, not a production network-provisioning platform.

1.2 Purpose

The purpose of the project is to replace unstructured request handling with a structured, trackable process. Its objectives are to:

- provide a catalog-based form for submitting network service requests and capture the required requester and connection details;
- reduce incomplete submissions with required fields and show the Existing ID field only when it is relevant;
- populate requester information automatically when the user data is available;
- store request information in a custom table and start an approval workflow;
- track fulfilment work in a separate task table; and
- generate a request-created email and a resolution email, demonstrating approvals, record relationships and notifications.

2. IDEATION PHASE

2.1 Problem Statement

Organizations regularly receive network service requests from employees and other users. When these are handled without a structured process, the following problems occur:

- incomplete request information and repeated manual data entry;
- difficulty tracking requests and their status;
- delays caused by manual approval handling;
- confusion over responsibility and task assignment;
- limited visibility of approval and processing progress;
- no timely acknowledgement or resolution notification; and
- inconsistent validation of the information users submit.

2.2 Empathy Map Canvas

The empathy map below describes the main user, the **Requester** (an employee who needs a network service). It is based on analysis of the project and is an assumption to validate with real users; no interviews or surveys were carried out.

Quadrant	Requester
Says	• "I need a new network connection / a change to my connection." • "Did my request go through?" • "What is my request number?"
Thinks	• Have I filled in everything that is needed? • Will it be approved quickly? • Who is handling my request?

Quadrant	Requester
Does	• Opens the Service Portal and the Network Request form. • Enters details, chooses New or Existing, and submits. • Waits, then asks for status updates.
Feels	• Uncertain whether the request was received. • Frustrated by repeated entry and delays. • Relieved when acknowledged and when resolved.
Pains	• Incomplete or repeated information, no visibility of status, manual approval delays, no notifications.
Gains	• One structured form, a request number, an acknowledgement with the expected timeframe, linked approval and task records, and a resolution notice.

2.3 Brainstorming

The table below analyses the solution approaches that were considered. It is a reasoned comparison derived from the project objectives, not a record of a workshop.

Approach	Idea	Outcome
Email / spreadsheet	Receive requests by email or phone and track them in a spreadsheet.	Not chosen: unstructured data, repeated entry, hard to track, no linked approval or notices.
Standalone custom web application	Build a separate front end, back end and database.	Not chosen: needs separate hosting, database and authentication; more effort than the objectives require.
ServiceNow catalog item with flows and custom tables	Use native platform features for the form, validation, workflow, approval, tasks and email.	Chosen: one platform covers every stage with low-code configuration.

Feature ideas generated from the objectives were then sorted into implemented features and future work:

Implemented	Future scope (not implemented)
Conditional Existing ID field; required fields; requester auto-population; request record creation; approval with Approved / Rejected updates; task table; request-created and resolution emails.	Automatic task creation after approval; provisioning-tool integration; dashboards and service-level targets; more request categories; stronger validation; email delivery monitoring.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The customer journey map follows the Requester through five stages. A full poster version is provided as a separate document; its content is summarised here. It is based on project configuration and development testing, not on customer interviews.

Stage	Main steps	Positive moments	Negative moments / opportunities
Entice	Need a network service; open the Service Portal (/sp); find the Network Request item.	One entry point in the portal.	Opportunity: guide requesters to the right catalog item.
Enter	Open the form (defaults New / UPI); select requester; enter details; submit (Existing ID if Existing).	Defaults, auto-filled user details and conditional Existing ID reduce typing.	No custom ten-digit mobile check; Phone Number stays empty if the profile has none.
Engage	Record created; acknowledgement email; approval requested; decision applied; task worked.	Record, approval and status updates were observed in testing.	Task creation is manual; rejection branch not freshly tested.
Exit	Task Completed; resolution email generated; requester informed.	Resolution email generated and previewed in the Outbox.	Outbox state Ready does not prove inbox delivery.

Stage	Main steps	Positive moments	Negative moments / opportunities
Extend	Request and task records retained; request number available for reference.	Approval Status and Work Status are kept separate.	Opportunity: dashboards, service-level targets, more request types.

3.2 Solution Requirement

Functional requirements

FR No.	Functional requirement (epic)	Summary
FR-1	Network Service Request Submission	Open the Network Request item in the Service Portal; capture requested-for, mobile number, connection type, amount, payment mode and address; defaults New and UPI.
FR-2	Requester Information Handling	User Name and Email update when the selected user changes; Phone Number fills when the profile has a mobile number.
FR-3	Form Validation	Existing ID visible and mandatory only for Existing connections; required fields prevent incomplete submission.
FR-4	Request Record Creation	Flow creates a Database Tables record with Approval Status = Requested and Database Number from the requested item.
FR-5	Request Acknowledgement	Email 'Request has been Created' with the request number and two business working days.
FR-6	Approval Management	Approval requested; Approved sets ITIL User and Approved; Else sets Abraham Lincoln and Rejected.
FR-7	Task Tracking	Network Task Table with Work Status, linked to the request record (created manually at present).
FR-8	Resolution Notification	Email 'Request has been Resolved' when Work Status changes to Completed.

Non-functional requirements

NFR No.	Requirement	Summary
NFR-1	Usability	Single portal form with defaults, auto-population and a conditional field.
NFR-2	Security	Application scope, application roles, ACLs and platform authentication (development-level; least-privilege review needed).
NFR-3	Reliability	Activated Flow Designer flows on a platform-managed database; wider regression testing recommended.
NFR-4	Performance	Server-side processing; no load or response-time figures measured.
NFR-5	Availability	Depends on the ServiceNow-hosted PDI; no production availability claimed.
NFR-6	Scalability	Catalog, flows and tables can be extended; not tested with realistic volumes.

3.3 Data Flow Diagram

The system is the ServiceNow scoped application. Its data is stored in two custom tables and the platform approval table. Figure 3.1 shows the context (Level 0) diagram and Figure 3.2 the Level 1 diagram.

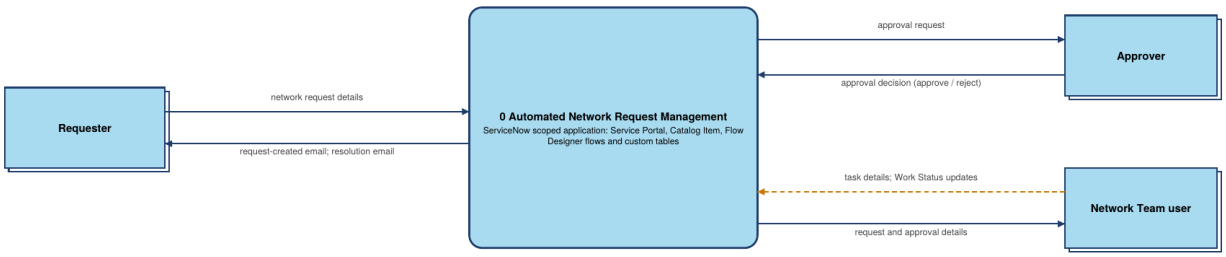


Figure 3.1: DFD Level 0 (context diagram)

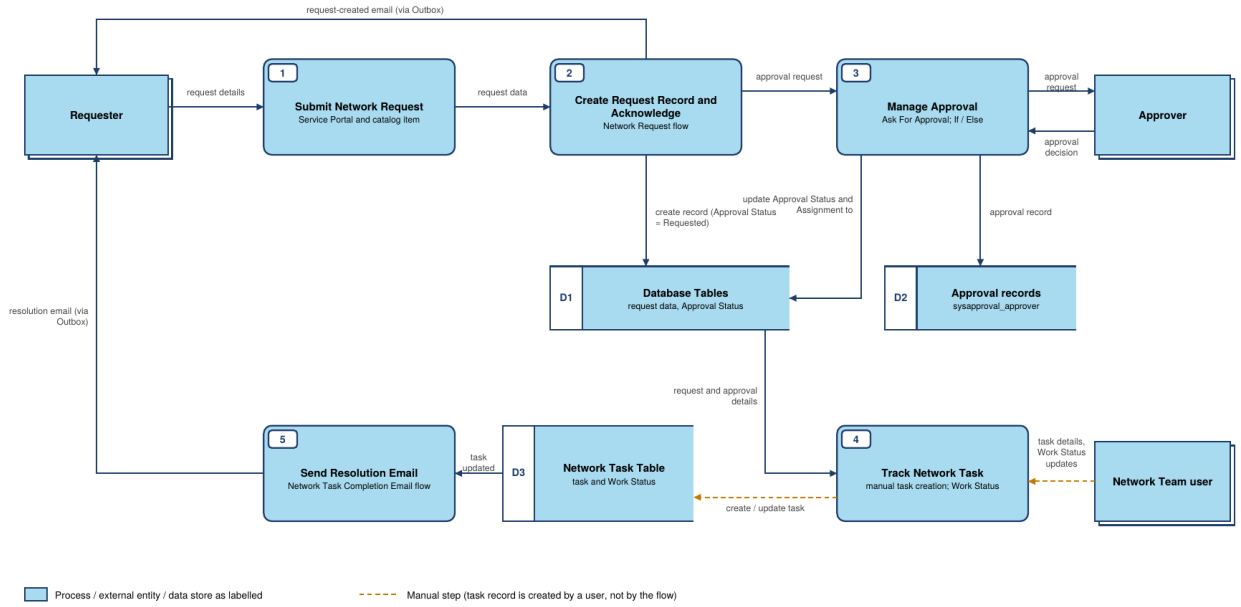


Figure 3.2: DFD Level 1 (dashed amber lines show the manual step)

3.4 Technology Stack

All layers use native ServiceNow capabilities. The stack is summarised below; Figure 3.3 shows the architecture and the components that are not implemented.

Layer	Technology
User interface	ServiceNow Service Portal (/sp), Service Catalog, catalog item, variables, variable set, Catalog UI Policy
Workflow engine	ServiceNow Flow Designer: Get Catalog Variables, Create Record, Send Email, Ask For Approval, If/Else, Update Record
Task processing	Custom Network Task Table and the Network Task Completion Email flow
Data storage	ServiceNow platform-managed database with two custom tables
Approval	ServiceNow approval records (sysapproval_approver) shown through a related list
Email	ServiceNow Send Email action and Outbox
Security	Scoped application, application roles, ACLs, platform authentication
Runtime	ServiceNow-hosted Personal Developer Instance
Not implemented	External database, external APIs, provisioning integration, machine learning, file storage, Docker/Kubernetes, production high availability, load testing

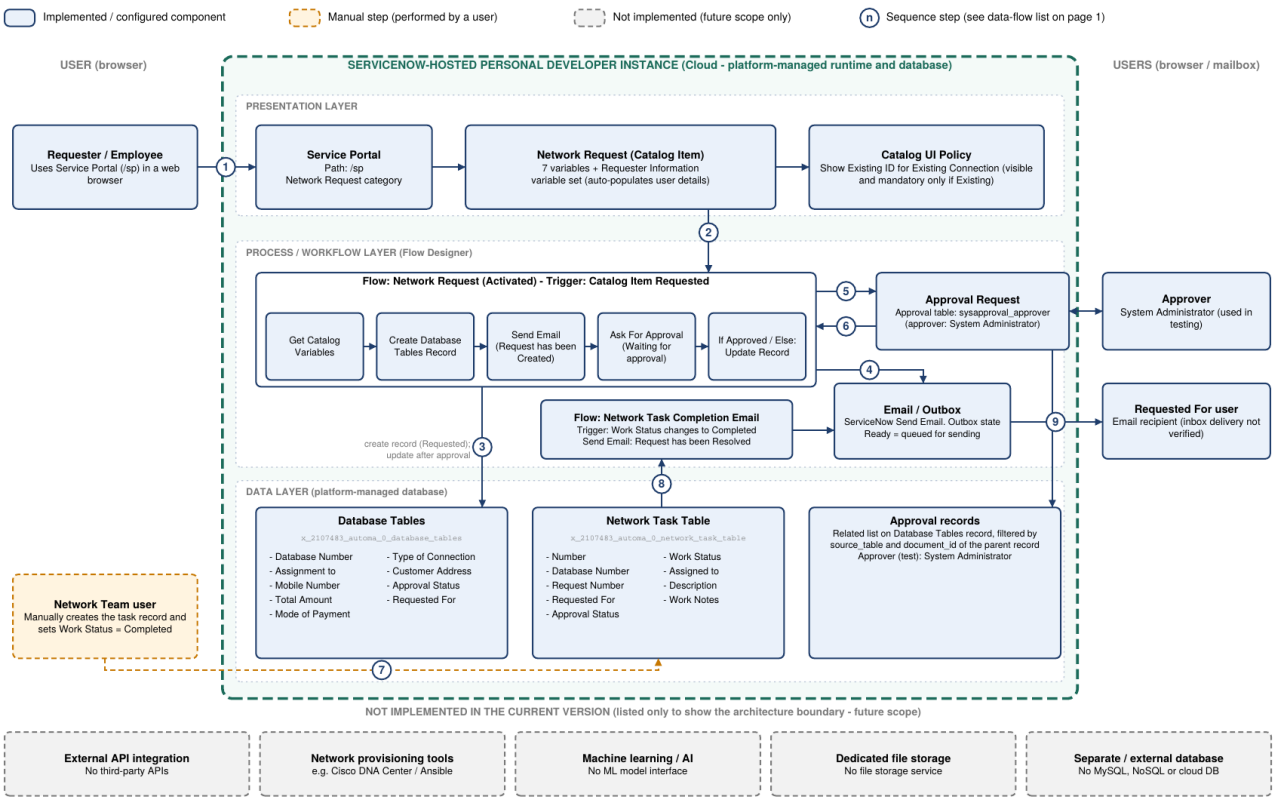


Figure 3.3: Technical architecture (implemented components and components not implemented)

4. PROJECT DESIGN

4.1 Problem Solution Fit

The Problem-Solution Fit canvas checks that the solution fits the customer's problem. A full canvas is provided as a separate document; the key points are:

Canvas element	Summary for this project
Customer segments	Requesters (employees and other users), approvers, Network Team members and administrators.
Customer constraints	Users must know the required details; approval depends on an approver; access to the portal and email is needed; the prototype runs in a developer instance.
Available solutions	Email, phone or spreadsheets: quick to start, but incomplete details, repeated entry and poor tracking.
Jobs-to-be-done / problems	Submit a complete request and know it was received; decide on requests; track tasks; be told when resolved.
Problem root cause	No structured form, manual approvals and tasks without linked records, no automatic notices, inconsistent checking.
Behaviour	Typical current behaviour is assumed (not surveyed): details sent by email or phone, then status chased.
Triggers and emotions	A new connection need; acknowledgement, approval and resolution notices. Intended shift from uncertain to confident (not measured).
Your solution	ServiceNow catalog item, Network Request flow, Network Task Table and completion email flow.
Channels	Online: Service Portal, flow emails, approval records. Offline: none built in.

4.2 Proposed Solution

Parameter	Description
Problem statement	Unstructured network request handling causes incomplete data, manual approvals, poor tracking and missing notices.
Idea / solution	A ServiceNow scoped application that organizes the request, approval, task and notification lifecycle (a prototype).
Novelty / uniqueness	Not a new technology; native platform features are combined for one process, with Approval Status and Work Status kept separate and two-stage emails.
Social impact / customer satisfaction	Expected: less repeated typing, fewer incomplete requests, visible status and timely notices. Satisfaction has not been measured.
Business model	Academic prototype with no revenue model; internal efficiency value is possible. Production use would need ServiceNow licensing and a cost estimate.
Scalability	New items, categories, flows and tables can be added; not load tested.

4.3 Solution Architecture

The architecture runs entirely inside a ServiceNow-hosted instance. The requester uses the Service Portal; the Network Request flow creates the request record, sends the acknowledgement email and asks for approval; the decision updates the record; a team member manually creates the task record; and the Network Task Completion Email flow sends the resolution email.

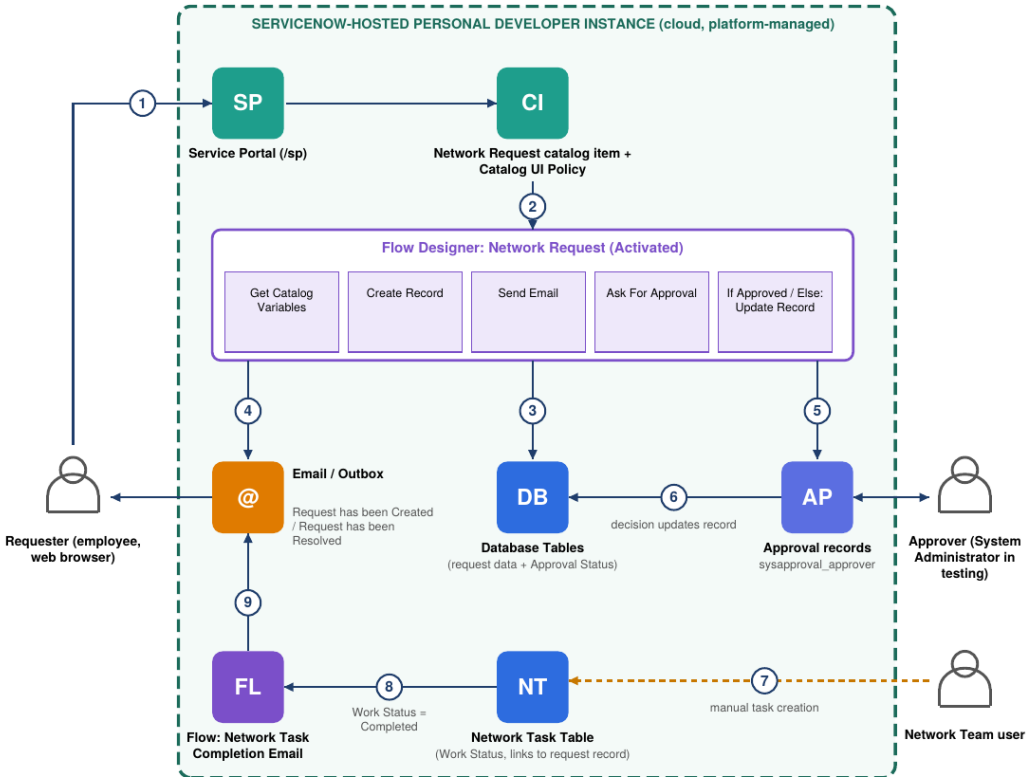


Figure 4.1: Solution architecture and data flow (steps 1 to 9)

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The product backlog uses the project's user stories. Story points are proposed relative estimates (1 to 5) and the sprint grouping is a proposed plan; sprint dates, completed points and team members were not recorded and are not stated.

Sprint	Planned stories	Story points
Sprint-1	USN-1 to USN-5: request submission, defaults, requester information, conditional Existing ID, required fields	12
Sprint-2	USN-6, USN-8 to USN-11: acknowledgement email, approval request, approve, reject, approval visibility	14
Sprint-3	USN-7, USN-12 to USN-14: resolution email, task creation, Work Status updates, security and access	13
Backlog (future)	USN-15: automatic task creation after approval (not implemented)	5 (not counted)
Total planned	Sprint-1 to Sprint-3	39

The planned velocity is (12 + 14 + 13) ÷ 3 = 13 story points per sprint. Average velocity per day is velocity ÷ sprint duration; it can be calculated once sprint durations and completed points are recorded. The planned burndown falls from 39 points to 27 after Sprint-1, 13 after Sprint-2 and 0 after Sprint-3; no actual burndown data is available.

6. FUNCTIONAL AND PERFORMANCE TESTING

Testing was manual functional testing in the developer instance, including a screen recording of the complete process. No automated test tools were used.

Functional testing results

Section	Total cases	Not tested	Fail	Pass
Catalog Request Form	3	0	0	3
Requester Information	3	0	0	3
Approval Flow	3	1	0	2
Task Tracking	1	0	0	1
Email Notifications	2	0	0	2
Totals	**12**	**1**	**0**	**11**

The counts are of 12 documented scenarios, not a formal test suite. Passing means the behaviour was observed in the development instance. The scenarios were:

- Request submission creates a request and a Database Tables record with the submitted values (observed in a later test; an earlier run showed missing catalog-variable warnings).
- Existing ID is visible and mandatory for Existing, and hidden for New.
- User Name and Email update when the selected user changes; Phone Number fills when the profile has a mobile number.
- A new request enters Requested status with an approval record; approval sets Approved and ITIL User.
- A Network Task Table record was created manually and linked to the request.
- The request-created email (RITM0010011, two business working days) and the resolution email were generated in the Outbox with state Ready; inbox delivery was not verified.
- **Not tested:** the rejection branch (Rejected and Abraham Lincoln) is configured but has no fresh execution record.

6.1 Performance Testing

Performance testing was not carried out. No response-time, throughput, concurrency or load-test figures exist, and none are claimed. The application runs in a Personal Developer Instance, which is not intended for performance benchmarking. Design choices that help performance are the short seven-variable form, server-side processing in Flow Designer, and a narrow trigger on the completion flow (Work Status changes to Completed, run for each unique change). Performance testing with realistic request volumes is listed as future work.

7. RESULTS

7.1 Output Screenshots

The screenshots were captured from a screen recording of the running application (recorded 26 October 2026). The recipient email address in the Outbox image is hidden.

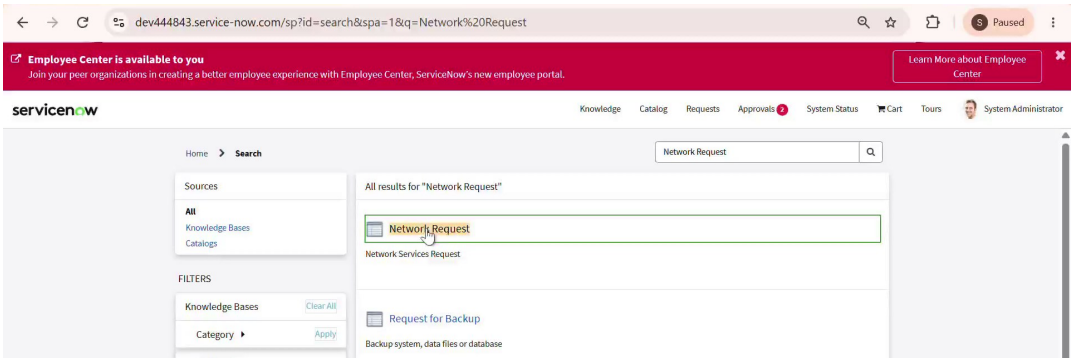


Figure 7.1: Service Portal (/sp) search result showing the Network Request catalog item

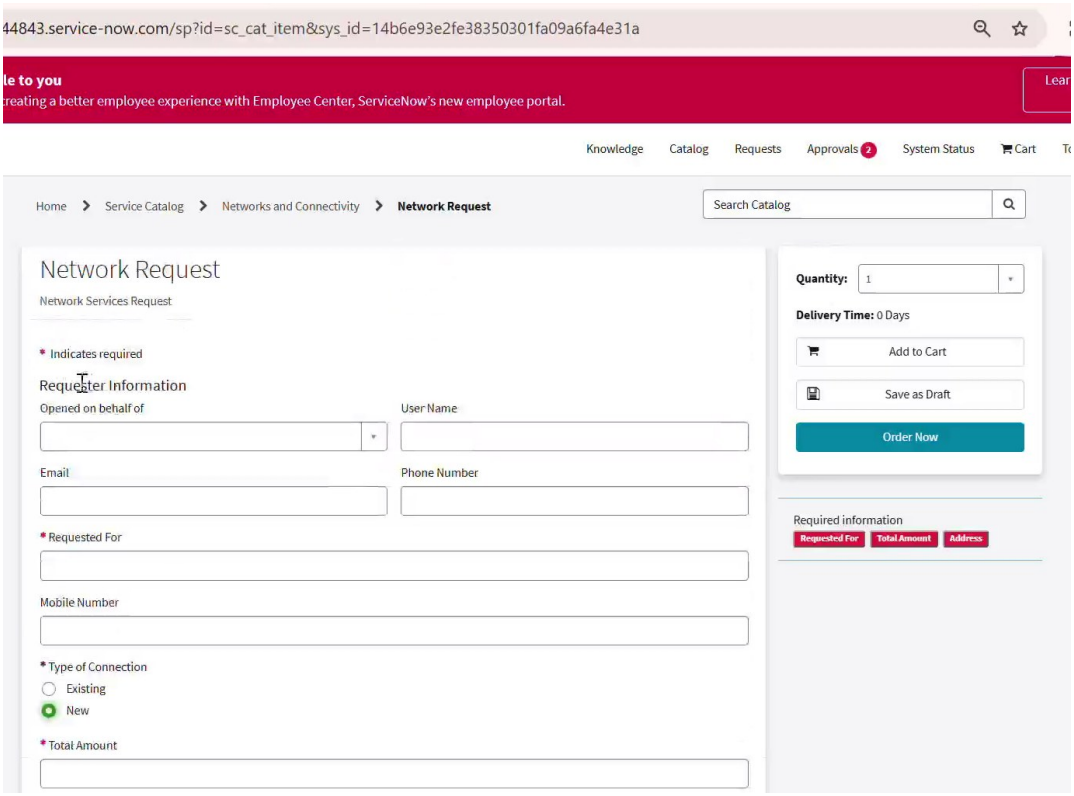


Figure 7.2: Network Request form with New selected; Existing ID is hidden

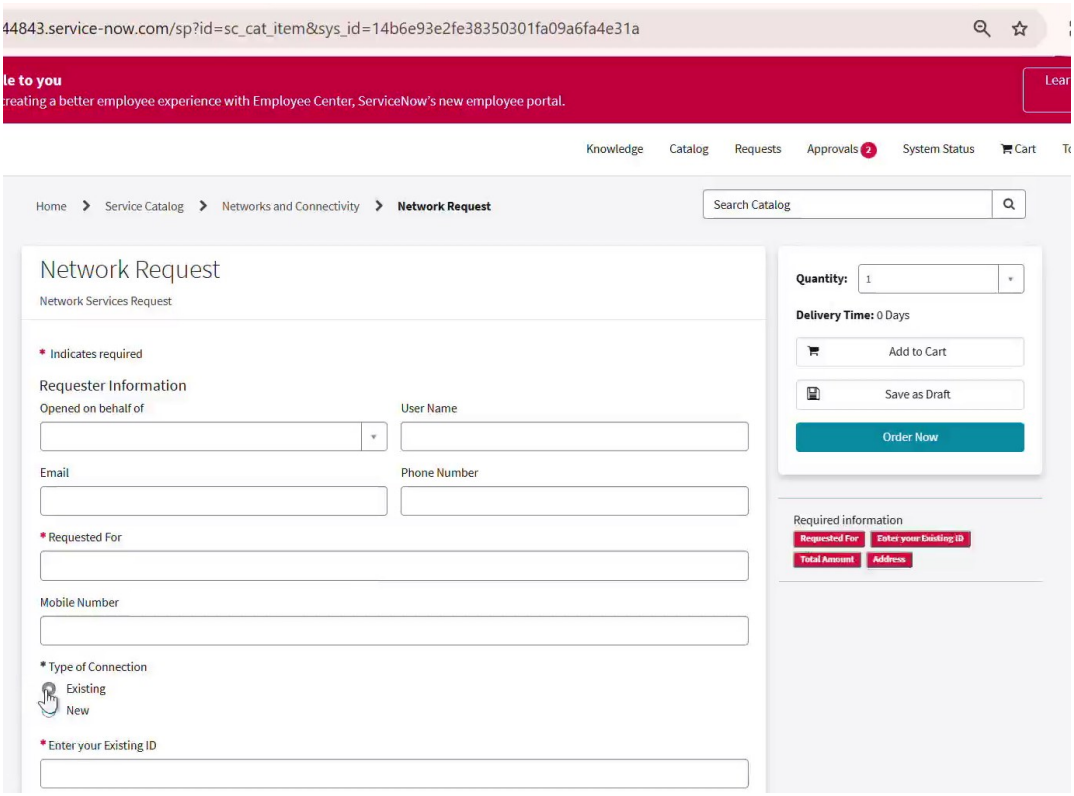


Figure 7.3: Existing selected; 'Enter your Existing ID' appears and is mandatory

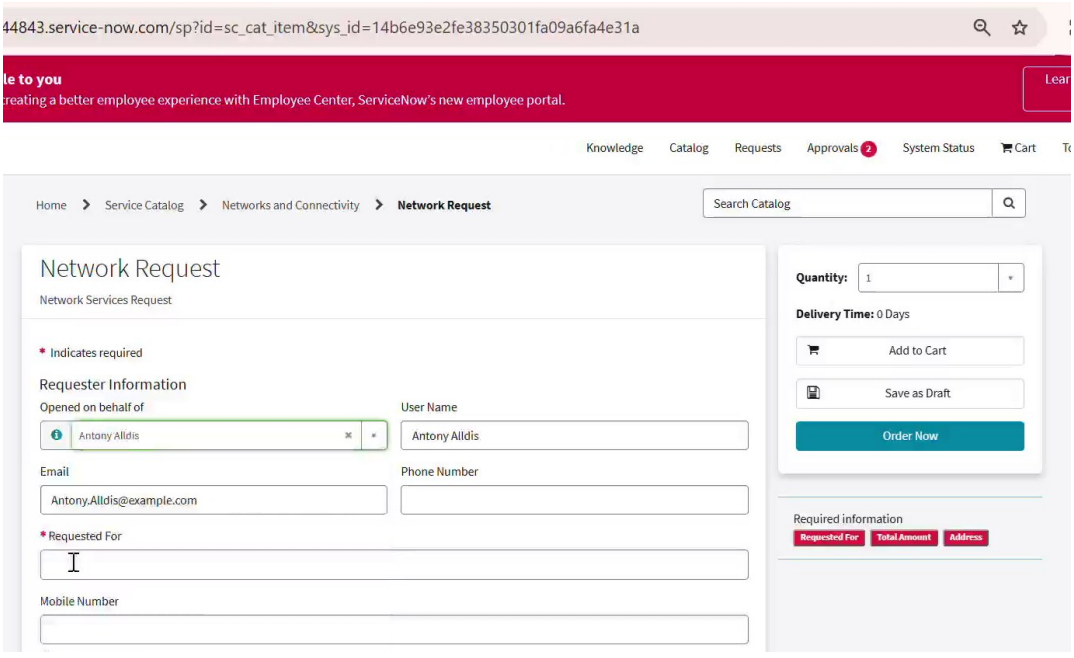


Figure 7.4: Requester Information filled by selecting a user (Phone Number is empty for this user)

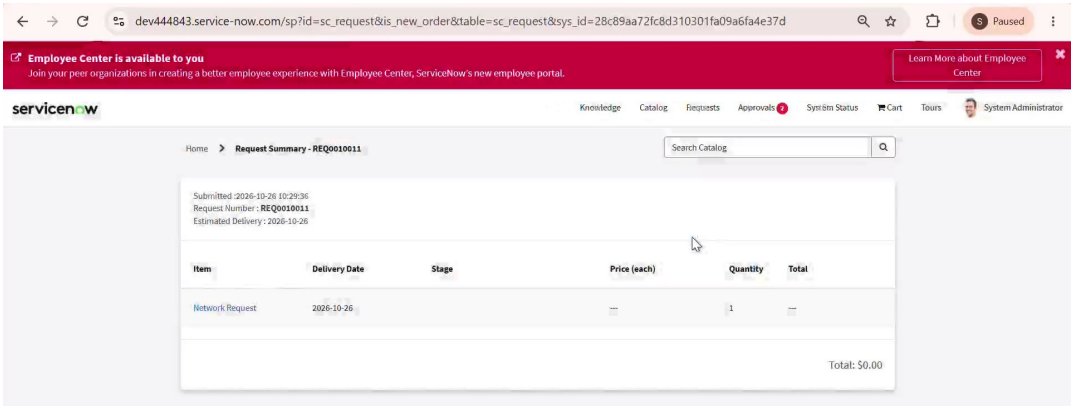


Figure 7.5: Request Summary after ordering (REQ0010011)

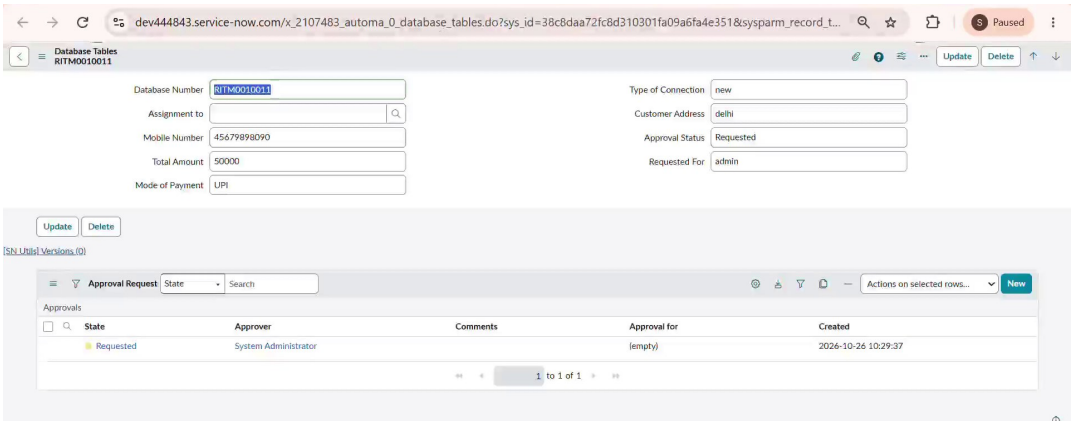


Figure 7.6: Database Tables record with Approval Status = Requested and its approval record

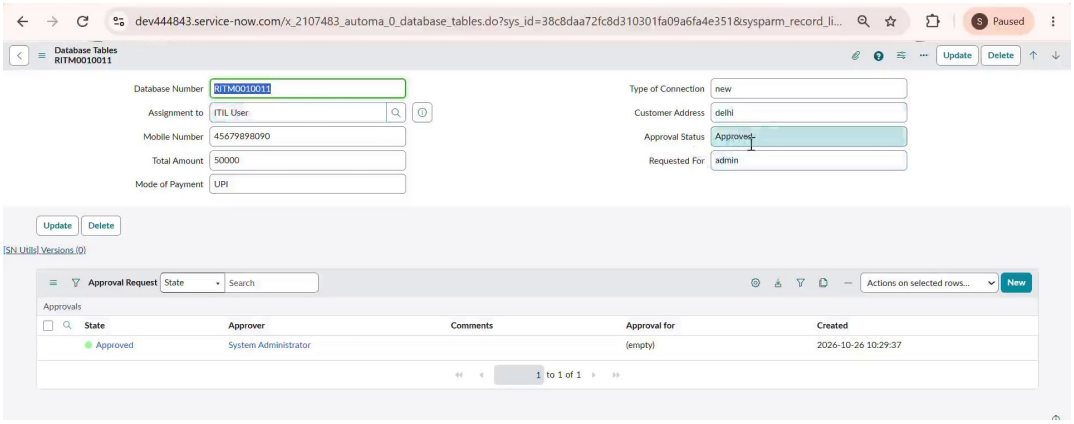


Figure 7.7: After approval: Approved, Assignment to = ITIL User

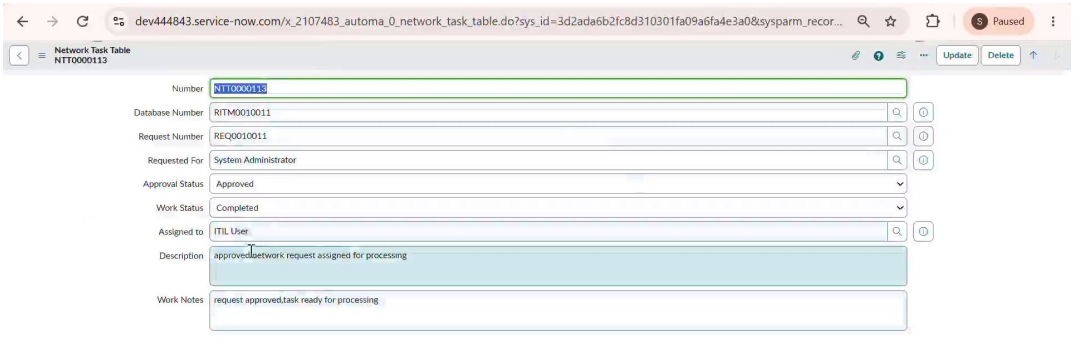


Figure 7.8: Network Task Table record with Work Status = Completed

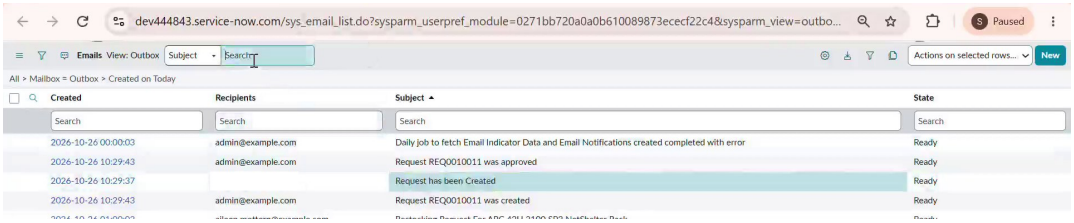


Figure 7.9: Outbox entry 'Request has been Created' (State Ready); recipient hidden

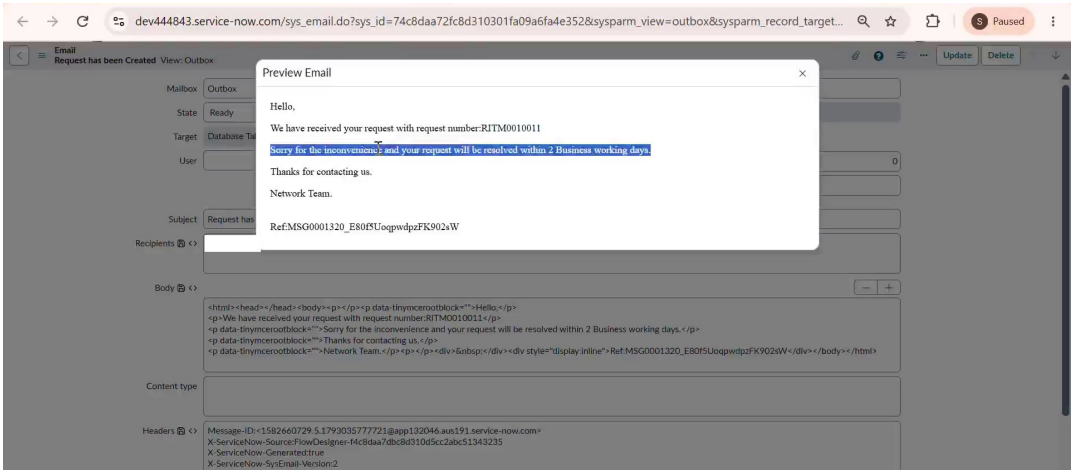


Figure 7.10: Preview of the request-created email (RITM0010011, 2 business working days; recipient hidden)

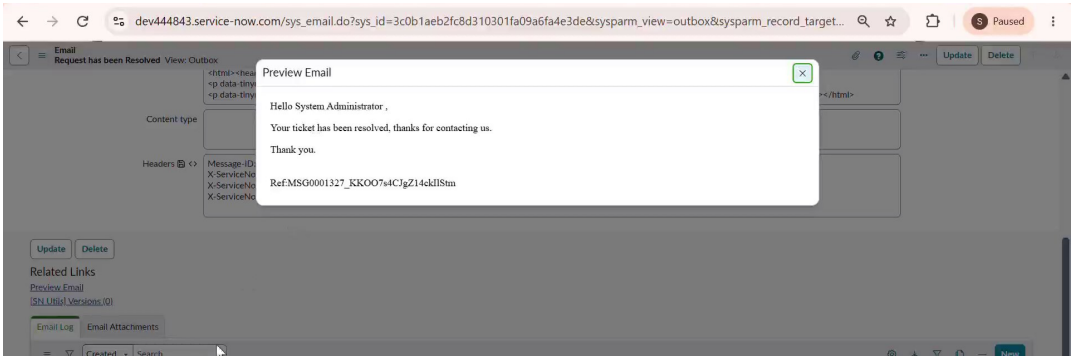


Figure 7.11: Preview of the 'Request has been Resolved' email

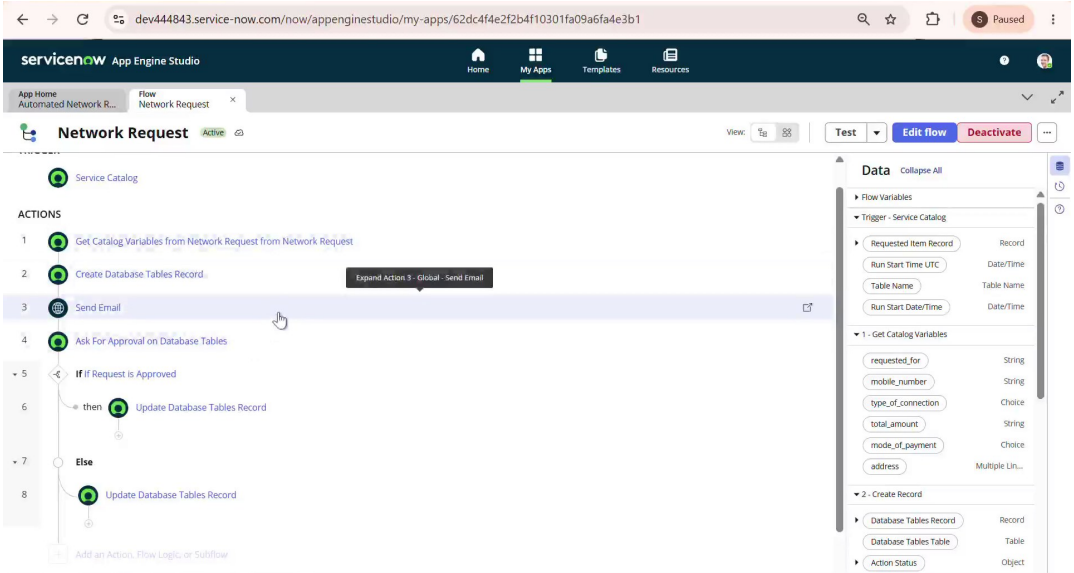


Figure 7.12: Network Request flow in Flow Designer

8. ADVANTAGES & DISADVANTAGES

Advantages

- Structured data capture through one catalog form with required fields and a conditional Existing ID field.
- Requester details can be auto-populated, reducing repeated typing.
- Every request is stored with a request number, and approval records are linked to the request record.
- Approval Status and Work Status are kept separate, making progress easier to understand.
- Acknowledgement and resolution emails keep the requester informed.
- Built on a single platform with low-code configuration: no separate server, database or front-end framework.
- Extendable with new catalog items, flows and tables.

Disadvantages and limitations

- It is a prototype in a developer instance, not a production system.
- Network Task Table records are created manually; the flow does not create them.
- The rejection branch is configured but has not been freshly tested.
- An Outbox state of Ready shows queuing, not inbox delivery.
- No custom ten-digit validation for Mobile Number, and Mobile Number is not marked as required on the form in the recording.
- An earlier run showed missing catalog-variable warnings; a later test succeeded, but wider regression testing is needed.
- ACLs are development-level and need a least-privilege review.
- No performance or load testing was done.
- It depends on the ServiceNow platform and, for production, on suitable licensing.

9. CONCLUSION

The project shows that a structured network request lifecycle can be built entirely with native ServiceNow capabilities. A Service Catalog item captures complete request details, a Flow Designer flow stores the request, acknowledges it and routes it for approval, custom tables keep the request and task data separate, and a second flow notifies the requester when work is completed. Testing in the developer instance observed the main behaviours working (11 of 12 documented scenarios), including conditional validation, approval updates and both emails.

The prototype also has clear limits: task creation is manual, the rejection branch needs a fresh test, email delivery was verified only to the Outbox, and no performance testing has been done. With those limits addressed, the application provides a sound foundation for a fuller network request management service.

10. FUTURE SCOPE

- Create the Network Task Table record automatically after approval.
- Integrate with network provisioning tools such as Cisco DNA Center or Ansible.
- Use Integration Hub for external workflow integrations.
- Add dashboards for request volume, approval time and task completion.
- Support more request categories, such as VPN access or device provisioning.
- Introduce service-level targets and detailed audit reporting.
- Improve email delivery monitoring and failure handling.
- Add stronger field-level validation and test coverage.
- Review ACLs for least-privilege access.
- Run performance tests with realistic request volumes.

- Define deployment, update-set and rollback procedures for moving the application to another instance.

11. APPENDIX

Source Code (if any)

The application was built with ServiceNow low-code configuration; there is no separate source-code repository. The only script is the related-list filter that shows approval records on the Database Tables record:

```
(function refineQuery(current, parent) {
    current.addQuery('source_table', parent.getTableName());
    current.addQuery('document_id', parent.sys_id);
})(current, parent);
```

Dataset Link

Not applicable. The application uses no external dataset; its data is created by users through the catalog form.

GitHub & Project Demo Link

GitHub repository: not provided.
Project demo link: not provided. A screen recording of about 15 minutes (26 October 2026) covering the full process is held by the team.